Assignment -9.2

Name: Akshaya Nemalipuri
Batch: 37
Roll No: 2303A2441
Lab: 9.2

Task Description -1 (Documentation – Function Summary Generation)
Task:
Use AI to generate concise functional summaries for each Python function in a given script.
Instructions:
• Provide a Python script to the AI.
• Ask the AI to write a short summary describing the purpose of each function.
• Ensure summaries are brief and technically accurate.
• Do not include code implementation details.
Expected Output -1:
A Python script where each function contains a clear and concise summary explaining its purpose.

Code:

```
#function.py
#2 function.py
#with docstring
def add_numbers(a, b):
    """
    This function takes two numbers as input and returns their sum.

    Parameters:
    a (int or float): The first number to be added.
    b (int or float): The second number to be added.

    Returns:
    int or float: The sum of the two input numbers.
    """
```

```python
    return a + b
# Example usage
if __name__ == '__main__':
    num1 = 5
    num2 = 10
    result = add_numbers(num1, num2)
    print(f"The sum of {num1} and {num2} is: {result}")
def fibonacci(n):
    """
    This function returns the Fibonacci sequence up to the nth number.

    Parameters:
    n (int): The number of Fibonacci numbers to generate.

    Returns:
    list: A list containing the Fibonacci sequence up to the nth number.
    """
    sequence = []
    a, b = 0, 1
    for _ in range(n):
        sequence.append(a)
        a, b = b, a + b
    return sequence
# Example usage
if __name__ == '__main__':
    n = 10
    result = fibonacci(n)
    print(f"The Fibonacci sequence up to the {n}th number is: {result}")
```

Summary’

`add_numbers(a, b)`

- **Purpose**: Takes two numbers as input and returns their sum
- **Parameters**: `a` and `b` (int or float)
- **Returns**: The sum of the two numbers

2. `fibonacci(n)`

- **Purpose**: Generates and returns the Fibonacci sequence up to the nth number
- **Parameters**: `n` (int) - the number of Fibonacci numbers to generate
- **Returns**: A list containing the Fibonacci sequence

Example Outputs:

- `add_numbers(5, 10)` → `15`
- `fibonacci(10)` → `[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]`

Both functions include detailed docstrings explaining parameters and return types, and have example usage blocks that demonstrate how to call them when the script is run directly.

Task Description -2 (Documentation – Logical Explanation for

Conditions and Loops)

Task:

Use AI to document the logic behind conditional statements and loops in

a Python program.

Instructions:

• Provide a Python program without comments.

• Instruct AI to explain only decision-making logic and loop

behavior.

• Skip basic syntax explanations.

Expected Output -2:

Python code with clear explanations describing the logic of conditions

and loops

Code:-

```
# This program demonstrates the use of conditions and loops in Python.
```

```python
def main_with_docstring():
    """
    This function demonstrates the use of conditions and loops in Python.
    It checks if a number is positive, negative, or zero and then uses a loop to print numbers from 1 to 5.
    """
    number = int(input("Enter a number: "))

    # Check if the number is positive, negative, or zero
    if number > 0:
        print("The number is positive.")
    elif number < 0:
        print("The number is negative.")
    else:
        print("The number is zero.")

    # Use a loop to print numbers from 1 to 5
    print("Numbers from 1 to 5:")
    for i in range(1, 6):
        print(i)
```

Task Description -3 (Documentation – File-Level Overview)

Task:

Use AI to generate a high-level overview describing the functionality of

an entire Python file.

Instructions:

• Provide the complete Python file to AI.

• Ask AI to write a brief overview summarizing the file's purpose

and functionality.

• Place the overview at the top of the file.

Expected Output -3:

A Python file with a clear and concise file-level overview at the

Beginning.

Code:

```python
# This file contains a simple function that adds two numbers together and returns the result.

def add_numbers(a, b):

    """

    This function takes two numbers as input and returns their sum.


    Parameters:

    a (int or float): The first number to be added.

    b (int or float): The second number to be added.


    Returns:

    int or float: The sum of the two input numbers.

    """

    return a + b
```

Summary

File Overview

Purpose: Simple utility function for basic arithmetic operations

Functionality:

add_numbers(a, b) - Adds two numbers and returns their sum

Accepts integers or floats as input

Returns the sum as an integer or float

Well-documented with a comprehensive docstring

Use Case: Provides a straightforward reusable function for adding two numeric values together.

Task Description -4 (Documentation – Refine Existing

Documentation)

Task:

Use AI to improve clarity and consistency of existing documentation in

Python code.

Instructions:

• Provide Python code containing basic or unclear comments.

• Ask AI to rewrite the documentation to improve clarity and

consistency.

• Ensure technical meaning remains unchanged.

Expected Output -4:

Python code with refined and improved documentation that is clear and consistent.

Code:-

```
def calculate_area_docstring(radius):
    """
    Calculate the area of a circle given its radius.

    Parameters:
    radius (float): The radius of the circle.

    Returns:
    float: The area of the circle calculated using the formula A = π *
r^2.
    """
    import math
    area = math.pi * (radius ** 2)
    return area
#example usage
if __name__ == '__main__':
    radius = 5
    area = calculate_area_docstring(radius)
    print(f"The area of a circle with radius {radius} is: {area}")
```

Task Description -5 (Documentation – Prompt Detail Impact Study)

Task:

Study the impact of prompt detail on AI-generated documentation quality.

Instructions:

Create two prompts: one brief and one detailed.

• Use both prompts to document the same Python function.

• Compare the generated outputs.

Expected Output -5:

A comparison table highlighting differences in completeness, clarity, and accuracy of documentation.

Code:-

```
#Create two prompts: one brief and one detailed.Use both prompts to
document the same Python function. the generated outputs.

#brief prompt

def calculate_area(radius):

    """

    Calculate the area of a circle given its radius.


    Parameters:

    radius (float): The radius of the circle.


    Returns:
```

```
    float: The area of the circle calculated using the formula A = π *
r^2.

    """

    import math

    area = math.pi * (radius ** 2)

    return area

#detailed prompt

def calculate_area_docstring(radius):

    """

    Calculate the area of a circle given its radius.


    Parameters:

    radius (float): The radius of the circle.


    Returns:

    float: The area of the circle calculated using the formula A = π *
r^2.

    """

    import math

    area = math.pi * (radius ** 2)

    return area

#example usage

if __name__ == '__main__':

    radius = 5

    area = calculate_area_docstring(radius)

    print(f"The area of a circle with radius {radius} is: {area}")
```